

course: OOP (Object-Oriented Programming)

artifact: Mega Worksheet + Study Guide + Rubric

coverage:

- Stacks & Queues (ADT mindset refresher)
- Overloading vs Overriding
- Constructors & Initialization Lists
- struct vs class
- Operator Overloading
- friend
- Composition (intro, not yet the full assignment) due:

day: Monday the 16th

time: 11:00 AM

in\_class\_support:

day: Wednesday

note: Questions encouraged. Panic discouraged.

tone: Academically serious, emotionally sarcastic

philosophy: |

OOP is not about inheritance.

It is about designing types that are hard to misuse.

## ❓❓ OOP Worksheet & Study Guide

Making Your Own Types Feel Like the Language Meant Them to

Exist Due: 16 Feb 2026 by class time.

If this feels long, good.

If it feels unfinished, also good.

If you feel like there's "always more," congratulations — that feeling is the curriculum.

This document serves three purposes:

- . Homework
- . Study guide
- . A mild personality test for how you think about software design

Read the questions carefully. Many are intentionally phrased to trip up rote memorization.

## Part 0 — A Necessary Reality Check

Most people think:

OOP = inheritance

That's like saying:

Math = long division

Inheritance exists.

It is sometimes useful.

It is wildly overused.

This worksheet is about building abstract data types (ADTs) that behave like they belong in the language — not about summoning class hierarchies like Pokémon.

## Part 1 — Review, But With Consequences

### Stacks & Queues (ADT Reality Check)

Answer clearly and concisely.

. Compare array-based vs list-based implementations of stacks and queues:

memory layout: contiguous memory, Nodes scattered (holds pointers)

resizing behavior: occasional, dynamic allocation

cache friendliness: excellent, Poor(scattered memory)

(Yes, cache friendliness matters. No, you may not ignore it.)

. Why is `std::vector` a natural fit for a stack, but awkward for a queue?

Supports `push_back` and `pop_back`.

Requires `pop_front`, which makes things awkward.

. Define the invariant for:

a stack: only top element is accessible; push/pop affect the top

a queue: Only front element can be removed and only the back element can be added.

If your invariant takes more than one sentence, it's not an invariant — it's a confession.

## Part 2 — Overloading vs Overriding

(Same Word Root, Completely Different Beasts)

### Conceptual Distinction

Fill out the table:

| Feature | Overloading | Overriding |
|---------|-------------|------------|
|---------|-------------|------------|

|              |              |          |
|--------------|--------------|----------|
| Resolved at: | Compile Time | Run Time |
|--------------|--------------|----------|

|                       |    |     |
|-----------------------|----|-----|
| Requires inheritance? | No | Yes |
|-----------------------|----|-----|

|                     |     |     |
|---------------------|-----|-----|
| Same function name? | Yes | yes |
|---------------------|-----|-----|

|                      |    |    |
|----------------------|----|----|
| Same parameter list? | No | No |
|----------------------|----|----|

|                        |                   |               |
|------------------------|-------------------|---------------|
| Polymorphism involved? | Compile Time Poly | Run time Poly |
|------------------------|-------------------|---------------|

Then answer:

- . Why is overloading a compile-time convenience? Static Binding
- . Why is overriding a runtime contract? Dynamic Binding
- . Why do beginners confuse the two? Involves methods with the same name, use similar syntax, and often appear in the same class hierarchy.
- . Why is that confusion dangerous? Unexpected behavior and logic errors.

Hint: The compiler is not your therapist. It will not guess your intent.

## Part 3 — Constructors & Initialization

### Lists

(Where C++ Stops Holding Your Hand)

#### Initialization Lists or Else

Given:

```
class Widget {  
private:  
    const int id;  
    std::string name;  
  
public:  
    Widget(int id, std::string name);  
};
```

Answer:

- . Why must this constructor use an initialization list? Id is const.
- . What happens if you try to assign id inside the constructor body?  
Error. Const members are immutable
- Write the correct constructor. : id(id), name(name) {}
- . Name one other situation where initialization lists are required (research-lite).

Reference members

Saying “because the compiler told me to” is not an explanation.

# Copy Constructor vs Assignment Operator

Research + reasoning required.

- . When is the copy constructor invoked? The copy constructor is called when a new object is being created as a copy of another object
- . When is the assignment operator invoked? The assignment operator is called when an existing object is assigned the value of another existing object
- . Why do both exist? Copy constructor: Ensures proper initialization of a new object from another, handling deep copies if necessary.

Assignment operator: Ensures proper copy of data from one existing object to another, possibly releasing old resources first.

- . What subtle bugs appear if you confuse them? Double deletion / memory leaks, Dangling pointers, Resource corruption

“They both copy stuff” earns partial credit and a sigh.

## Part 4 — struct vs class

(Same Machine Code, Different Intent)

### Design Signal, Not Syntax Sugar

Answer:

- . What is the only language-level difference between struct and class?  
Struct: members and base classes are public by default

Class: members and base classes are private by default

.

Why does C++ even allow both? Historical / readability reasons

- . When does choosing struct communicate intent better than class? If a type is primarily data with public access and minimal behavior, struct communicates that clearly
- . Why does intent matter more than syntax in large systems? Large codebases rely on human readability and maintainability. Choosing struct vs class communicates how a type should be used

## Part 5 — Operator Overloading

(Where ADTs Start Feeling Real)

# Rules You Don't Get to Ignore

Research and explain:

. Why can't C++ overload:

.

::

Overloading (.) and (::) would break type safety and make the compiler unable to find members reliably.

sizeof

Overloading sizeof would require runtime behavior for something that is inherently compile-time

. Why should operator+ not mutate the left-hand operand? '+'

is arithmetic, not assignment. Mutating the left-hand operand

would violate user expectations and break referential

transparency.

. Why is operator<< almost never a member function? usually

ostream; must be a free function to support natural syntax

If your answer is "because that's how everyone does it," dig deeper.

## The "other / rhs" Trap (Yes, It's Intentional)

Given:

```
Point operator+(const Point& rhs) const;
```

Answer clearly:

. Which object owns this function? The function is a member of the class Point, so it belongs to the object on the left-hand side (the one calling it).

. What does rhs represent? (right-hand side) is the argument passed to the operator

. How can this function access rhs.x if x is private? the operator+ function can still read it

. What does this tell you about class-level vs object-level access?

Class-level access: Private/protected members are accessible to all instances of the same class, not just the current object (this).

Object-level access is a misleading mental model — private does not mean "only this object", it means "only members/functions of this class".

This question exists specifically to break incorrect mental models.

## Part 6 — friend: Controlled Violation of Privacy

### Friend or Design Smell?

Answer:

- . What does the friend keyword actually do? Allows access to private and protected data
- . Why is operator<< commonly declared as a friend? Its clean syntax
- . Why is excessive use of friend a red flag? Breaks encapsulation
- . Give one legitimate use case and one illegitimate one.

“Because it wouldn’t compile otherwise” is not a justification — it’s a symptom.

Legitimate: Non-member operator like operator<< or operator== needing access to

private members for efficiency or syntactic clarity.

Illegitimate: Making a random utility function a friend to bypass instead of

providing proper getter/setter.

## Part 7 — Core Programming Task

Design a Type, Not a School Assignment

### Build a Native-Feeling Point2D

You are designing a type, not checking boxes.

#### Required Features

Your Point2D class must:

- Use private data members

- Include at least three constructors:

  - default

  - parameterized

  - copy constructor

- Use initialization lists

Overload:

+

-

==

!= (without duplicating logic)

<<

Demonstrate const correctness

Avoid public getters unless you can justify them

### Required Usage (This Must Compile)

```
Point2D a(3, 4);
```

```
Point2D b(1, 2);
```

```
Point2D c = a + b;
```

```
Point2D d = a - b;
```

```
if (c == Point2D(4, 6)) {  
    std::cout << "Math still works.\n";  
}
```

```
std::cout << a << std::endl;
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Point2D {
```

```
private:
```

```
    int x;
```

```
    int y;
```

```
public:
```

```
    //Default
```

```
    Point2D() : x(0), y(0) {}
```

```
    //Parameterized
```

```
    Point2D(int xVal, int yVal) : x(xVal), y(yVal) {}
```

```
    //Copy
```

```
    Point2D(const Point2D& other) : x(other.x), y(other.y) {}
```

```
    //Addition
```

```
    Point2D operator+(const Point2D& rhs) const {
```

```

        return Point2D(x + rhs.x, y + rhs.y);
    }

//Subtraction

Point2D operator-(const Point2D& rhs) const {

    return Point2D(x - rhs.x, y - rhs.y);

}

```

Equality

```

bool operator==(const Point2D& rhs) const {

    return x == rhs.x && y == rhs.y;

}

```

//No duplicated logic

```

bool operator!=(const Point2D& rhs) const {

    return !(*this == rhs);

}

```

//Output

```

friend ostream& operator<<(ostream& os, const Point2D& p) {

    os << "(" << p.x << ", " << p.y << ")";

    return os;

}

```

```
};
```

```
int main() {
```

```
    Point2D a(3, 4);
```

```
    Point2D b(1, 2);
```

```
    Point2D c = a + b;
```

```
    Point2D d = a - b;
```

```
    if (c == Point2D(4, 6)) {
```

```
    cout << "Math still works.\n";  
  
}  
  
cout << a << endl;  
  
cout << d << endl;  
  
return 0;  
  
}
```

## Design Constraints (Read Carefully)

You may not:

- expose raw data publicly
- use inheritance
- overload operators with nonsense semantics

You should:

- minimize the public interface
- make misuse difficult
- prefer clarity over cleverness

## Part 8 — Composition (The Quiet MVP of OOP)

### Composition in Plain English

Composition means assembling behavior from parts, not becoming those parts. Or

more bluntly:

“Has-a” beats “is-a” most of the time.

### Inheritance vs Composition (No Dragons Yet)

 Inheritance First Instinct

```
class ColoredPoint : public Point2D {  
    Color color;
```

```
};
```

Problems:

Locked into Point2D forever  
Inherits everything, wanted or not  
Hard to evolve safely

## ✓ Composition Approach

```
class ColoredPoint {  
private:  
    Point2D position;  
    Color color;  
};
```

Benefits:

Clear ownership  
Modular behavior  
Fewer unintended side effects

## The Big Takeaway

Inheritance expresses identity.  
Composition expresses capability.

Most real systems care more about capability.

# Part 9 — Reflection (Yes, This Is Graded)

Answer honestly:

- . Does your Point2D feel like a built-in type? Yes
- . What design choice most contributed to that feeling? The value semantics
- . Which OOP concept currently feels overhyped? I honestly don't feel like any of it seems too overhyped at the moment.
- . Which one feels underrated? Friend operator, it is a long code that is very important to have memorized.
- . What part of this worksheet made you uncomfortable — and why? I would  
  
say overloading vs overriding, just because it took me a while to truly  
  
understand the slight differences between the two. Hoping I don't mess  
  
them up on the exam.



## Grading Rubric (100 Points)

### Conceptual Understanding — 25 pts

Overloading vs Overriding: 10  
Constructors & Init Lists: 10  
Struct vs Class (intent-based): 5

### Operator Overloading Design — 25 pts

Semantic correctness: 10  
Const correctness: 5  
Member vs friend choice: 5  
Minimal interface: 5

### Class Design & Encapsulation — 25 pts

Private data: 10  
Constructor quality: 5  
Initialization lists: 5  
Friend usage justification: 5

### Code Quality — 15 pts

Compiles cleanly: 5  
Readability & naming: 5  
Comments explain why: 5

### Reflection & Judgment — 10 pts

Honest reflection: 5  
Insight into tradeoffs: 5

## Final Instructor Note (Read This Twice)

That feeling of:

“There’s always something else... another way... something deeper...”

That’s not failure.

That’s abstraction having no ceiling.

The goal is not perfection.

The goal is better judgment.

## Deliverables

Make sure you have a folder called Assignments in your Github repo.

Create a folder in Assignments called H01 and place all of your documents in there. Generate a pdf of your solution and have the entire worksheet not just uploaded to Github, but also printed and brought to class Monday the 16th of February.

This really is a study guide as we will have an exam the week of the 16th. You have to use markdown to create your document. Why? It does syntax highlighting, and makes writing documents easy.

Markdown Getting Started

Markdown Cheatsheet

If you use code spaces or download VsCode, you can install plugins to let you convert markdown to pdf, and to allow you to print directly from VsCode.